

Task 1: Data Immersion & Wrangling

Sales_Dataset Analysis — ApexPlanet Software Pvt. Ltd.

Data Analytics Internship Deliverable

Prepared by: Brojo Mohan Dutta

Executive Summary

This report documents the data immersion, quality assessment, and cleaning pipeline applied to the ApexPlanet Sales_Dataset (1,000 transaction records, 12 source columns). The dataset was found to be largely clean: no full-row duplicates and no numerical outliers were detected. Two data quality issues required remediation — 2.0% missing Age values, 1.3% missing City values — and one structural issue was identified: nine records share the Order_ID “ORD100050” despite representing distinct transactions, indicating a broken primary key upstream. The cleaning pipeline below resolves all three issues, converts Order_Date to a proper datetime type, engineers five analysis-ready features, and exports a final dataset (cleaned_sales_dataset.csv) ready for exploratory analysis and dashboarding.

Step 1 — Data Access & Familiarization

The Sales_Dataset sheet contains 1,000 rows and 12 columns spanning transaction, customer, product, and revenue attributes. The data dictionary below documents each field's inferred type, meaning, and business relevance, established before any cleaning was performed.

Data Dictionary

Column	Data Type	Description	Business Relevance
Order_ID	String	Unique identifier assigned to each sales transaction	Primary key for joining transaction-level records; needed for order tracking and returns handling
Order_Date	Date	Calendar date the order was placed (YYYY-MM-DD)	Enables time-series trend analysis, seasonality detection, and monthly/yearly revenue reporting
Customer_ID	String	Unique identifier assigned to each customer	Enables customer-level aggregation: repeat-purchase rate, lifetime value, cohort analysis
Customer_Name	String	Name of the customer placing the order	Used for personalization and CRM lookups; not used in aggregate analysis (PII)
Age	Integer	Customer's age in years at time of purchase	Drives demographic segmentation and age-based marketing strategy
Gender	String	Customer's stated gender (Male / Female)	Supports demographic segmentation and targeted campaign design
City	String	City where the order was placed / delivered	Enables regional performance comparison and geographic expansion planning
Product	String	Name of the product purchased	Identifies best- and worst-selling SKUs for inventory and merchandising decisions
Category	String	Product category (e.g. Electronics, Grocery, Education)	Supports category-level revenue mix and cross-sell analysis
Quantity	Integer	Number of units purchased in the order	Core input for demand forecasting and inventory planning
Unit_Price	Float	Price per single unit of the product (INR)	Supports pricing strategy analysis and margin review
Total_Sales	Float	Total transaction value (Quantity × Unit_Price)	Primary revenue metric; basis for all sales KPIs and dashboards

Step 2 — Data Quality Assessment

Before any transformation, the raw dataset was profiled in Jupyter Notebook to surface missing values, duplicate records, formatting inconsistencies, and statistical outliers. The exact profiling code used is reproduced below.

Profiling Code

```
import pandas as pd
import numpy as np

RAW_PATH = "ApexPlanet_DataAnalytics_Dataset.xlsx"
df = pd.read_excel(RAW_PATH, sheet_name="Sales_Dataset")
print("Shape:", df.shape)
df.info()

# --- 2.1 Missing values ---
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
print(pd.DataFrame({"Missing_Count": missing, "Missing_%": missing_pct})
      .query("Missing_Count > 0"))

# --- 2.2 Duplicate records ---
print("Full-row duplicates:", df.duplicated().sum())
print("Rows sharing a non-unique Order_ID:",
      df["Order_ID"].duplicated(keep=False).sum())

# --- 2.3 Order_Date format check ---
print("Current dtype:", df["Order_Date"].dtype)
print(df["Order_Date"].head())

# --- 2.4 Outlier scan (IQR method) ---
numeric_cols = ["Age", "Quantity", "Unit_Price", "Total_Sales"]

def iqr_outliers(series):
    q1, q3 = series.quantile([0.25, 0.75])
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    return ((series < lower) | (series > upper)).sum()

for col in numeric_cols:
    print(col, "outliers:", iqr_outliers(df[col].dropna()))
```

Run in Jupyter Notebook 7.4.5 against the raw Excel source.

Findings Summary

Issue Category	Finding	Rows Affected
Missing values	Age missing	20 rows (2.0%)
Missing values	City missing	13 rows (1.3%)
Duplicates	Full-row duplicates	0 rows
Duplicates	Order_ID collisions (same ID, different transaction)	9 rows (Order_ID = ORD100050)
Date formatting	Order_Date stored as text, not datetime	1,000 rows (100%)
Outliers	IQR outliers across Age, Quantity, Unit_Price, Total_Sales	0 rows — all values within plausible business ranges

Key observations:

- Age and City are the only columns with missing values (2.0% and 1.3% respectively) — low enough for standard imputation rather than row deletion.
- Order_Date parses as plain text in a consistent YYYY-MM-DD pattern across all 1,000 rows, but pandas reads it as an object/string dtype until explicitly converted.
- Nine rows share Order_ID “ORD100050” but have different customers, products, and dates — this is a primary-key collision from the source system, not a duplicate transaction, so the fix is ID repair, not deletion.
- No IQR-based outliers were found in Age, Quantity, Unit_Price, or Total_Sales; all values sit within expected business ranges (Age 18–65, Quantity 1–10 units, Unit_Price ₹145.78–₹49,997.53).

Step 3 — Data Cleaning & Transformation

The full cleaning pipeline below is modular, heavily commented, and produces the final analysis-ready dataset. Each stage addresses one of the findings from Step 2.

End-to-End Cleaning Script

```
import pandas as pd
import numpy as np

# 1. LOAD DATASET
RAW_PATH = "ApexPlanet_DataAnalytics_Dataset.xlsx"
df = pd.read_excel(RAW_PATH, sheet_name="Sales_Dataset")
print(f"Rows loaded: {df.shape[0]}, Columns loaded: {df.shape[1]}")

# 2. HANDLE MISSING VALUES
# Age: median imputation - resistant to outliers, preserves distribution
df["Age"] = df["Age"].fillna(df["Age"].median())

# City: explicit "Unknown" label rather than inventing a location
df["City"] = df["City"].fillna("Unknown")

# 3. HANDLE DUPLICATES
# a) Drop exact full-row duplicates
df = df.drop_duplicates()

# b) Repair broken Order_ID primary key (same ID, different transactions)
# Reassign a unique surrogate suffix instead of deleting real sales
dupe_id_mask = df["Order_ID"].duplicated(keep=False)
df = df.reset_index(drop=True)
df["Order_ID_Original"] = df["Order_ID"]
df["Order_ID_Flag"] = np.where(dupe_id_mask, "Duplicate_ID_Reassigned", "OK")
df.loc[dupe_id_mask, "Order_ID"] = [
    f"{oid}-{i+1:02d}" for i, oid in enumerate(df.loc[dupe_id_mask, "Order_ID"])
]

# 4. STANDARDIZE Order_Date TO DATETIME
df["Order_Date"] = pd.to_datetime(df["Order_Date"], format="%Y-%m-%d",
                                  errors="coerce")

# 5. FEATURE ENGINEERING
df["Order_Year"] = df["Order_Date"].dt.year
df["Order_Month"] = df["Order_Date"].dt.month
df["Order_Month_Name"] = df["Order_Date"].dt.month_name()

age_bins = [17, 25, 35, 45, 55, 65]
age_labels = ["18-25", "26-35", "36-45", "46-55", "56-65"]
df["Age_Group"] = pd.cut(df["Age"], bins=age_bins, labels=age_labels,
                          include_lowest=True)
df["Avg_Price_Check"] = (df["Total_Sales"] / df["Quantity"]).round(2)

# 6. EXPORT ANALYSIS-READY DATASET
df.to_csv("cleaned_sales_dataset.csv", index=False)
print(f"Cleaned dataset exported. Final shape: {df.shape}")
```

Companion files: *ApexPlanet_Task1_Data_Wrangling.ipynb* (executable notebook) and *cleaned_sales_dataset.csv* (output).

Transformation Decisions & Rationale

Missing values

- Age (numeric): imputed with the column median rather than the mean, since the median is robust to skew and avoids fabricating a value influenced by any extreme ages.
- City (categorical): filled with an explicit “Unknown” label rather than the modal city — imputing a real city would fabricate geographic signal that was never captured.

Duplicates

- Full-row duplicates: none were found, but `drop_duplicates()` is retained in the pipeline as a defensive standard-practice step for future data loads.
- Order_ID collisions: the 9 affected rows were not deleted. Each was assigned a unique surrogate ID (suffix -01, -02, ...) and flagged via a new Order_ID_Flag column, preserving all transaction data while making the anomaly traceable for the source-system owner.

Date standardization

- Order_Date was converted with `pd.to_datetime(..., format="%Y-%m-%d", errors="coerce")` — specifying the exact format avoids ambiguous day/month parsing and `errors="coerce"` surfaces any unparseable value as NaT rather than silently failing.

Feature engineering

- Order_Year, Order_Month, and Order_Month_Name were derived from Order_Date to support monthly/yearly trend and seasonality analysis.
- Age_Group bins customers into five 10-year bands (18–25 through 56–65) for demographic segmentation in dashboards.
- Avg_Price_Check ($\text{Total_Sales} \div \text{Quantity}$) is retained as a QA field to flag any future record where Total_Sales does not reconcile with $\text{Quantity} \times \text{Unit_Price}$.

Output

The pipeline exports the final dataset to `cleaned_sales_dataset.csv` — 1,000 rows × 19 columns, zero missing values, Order_Date as a proper datetime type, and five new analysis-ready fields. This file is ready for exploratory data analysis, visualization, or loading into Tableau/Power BI for Task 2.

Conclusion

The ApexPlanet Sales_Dataset required lightweight but deliberate cleaning: standard imputation for two sparse fields, a targeted repair for a primary-key collision, datetime standardization, and five engineered features. No rows were discarded solely for convenience — every remediation decision favored preserving legitimate transaction data over blunt deletion. The dataset is now analysis-ready for Task 2 (Exploratory Data Analysis & Visualization).